

Analyzing Model Performance in Reduced Feature Spaces: A Study on CNN, FNN and K-Means

Emanuele Smisi 2278705

Giacomo Aroni 2266616

January, 2026

1 Introduction

This project analyzes and compares various supervised and unsupervised Machine Learning approaches applied to heterogeneous datasets, including both images and tabular data. The primary objective is to understand how different models and techniques perform across varying data types and tasks, evaluating their performance, generalization capabilities, with particular focus on the impact and behavior in the presence of dimensionality reduction.

In particular, the work focuses on three models and their related tasks :

- **Classification:** The first part addresses a visual recognition and classification problem, utilizing a Convolutional Neural Network (CNN) to classify fish species from RGB images.
- **Regression:** The second part focuses on a regression task using a used car dataset to predict vehicle prices via a Feed-forward Neural Network (FNN).
- **Clustering:** The third part covers an unsupervised learning problem, applying the K-Means algorithm to a customer segmentation dataset to evaluate how dimensionality reduction influences the quality and stability of the resulting clusters.

Each model implementation and results are reported in their related sections. In each implementation, dimensionality reduction is applied using two different techniques (PCA and Autoencoders) to analyze whether this approach yields advantages based on the specific task.

1.1 Methodological Framework

To address the different nature of the tasks involved in this project, we adopted a flexible experimental setup that combines multiple machine learning frameworks. The goal was not to favor a specific tool, but to select the most suitable framework for each task, balancing control, clarity, and development efficiency.

For the **Classification task**, **PyTorch** was chosen because it is flexible and allows to control the training process step-by-step. This made it easier to customize the architecture and understand exactly how the CNN learns to recognize patterns in the image data.

For the **Regression and Clustering tasks**, we relied on **TensorFlow/Keras**. This framework was selected because its design facilitates rapid model building and testing. This tool was ideal for handling our structured data, allowing us to focus on processing variables efficiently without needing the complex controls required for images.

Across all experiments, a **consistent pipeline** was followed. Each dataset was first preprocessed through cleaning, encoding of categorical variables, and feature scaling. When dimensionality reduction was required, both Principal Component Analysis (PCA) and Autoencoders were applied to reduce the data to the same latent dimensionality, ensuring a fair comparison. Model hyperparameters were selected using cross-validation, where possible, and performances were evaluated using task-appropriate metrics.

2 Dataset Overview

The robustness of the proposed architectures is evaluated across three distinct datasets, each specifically selected to align with the respective task of the model and reflect standard usage scenarios. Datasets were retrieved from [Kaggle](#) platform.

2.1 Large Scale Fish Dataset

The first part of our analysis utilizes the [Large Scale Fish Dataset](#), a collection designed for high-accuracy species classification. For the purpose of this project, we focused on the RGB image subset. It comprises nine distinct classes representing commercially relevant species: Gilt-Head Bream, Red Sea Bream, Sea Bass, Red Mullet, Horse Mackerel, Black Sea Sprat, Striped Red Mullet, Trout, and Shrimp.

2.2 Audi Used Car Dataset

For the regression task, we employed the [Audi Used Car Dataset](#), which provides structured information on vehicle sales. The goal is to predict the **Price** of a vehicle based on a set of heterogeneous attributes:

- **Categorical features:** model, transmission, fuelType.
- **Numerical features:** price (target), year, mileage, engineSize, tax and mpg (miles per gallon).

This dataset allows us to test the efficacy of Feed-forward Neural Networks (FNN) in capturing non-linear relationships within pricing dynamics.

2.3 Customer Segmentation Dataset

For the unsupervised learning segment, we utilized a [Customer Segmentation Dataset](#) from the automotive sector. This dataset is designed to identify latent consumer profiles through a mix of demographic and behavioral features, including:

- **Categorical features:** Gender, Married, Graduated, Profession, SpendingScore and Category
- **Numerical features:** CustomerID, Age, WorkExperience and FamilySize

Unlike the previous task, this dataset lacks a target label for the clustering phase, requiring the models to discover natural groupings within a multi-dimensional feature space. The primary challenge here lies in the effective integration of ordinal features (like *Spending Score*) and binary features, testing the ability of PCA and Autoencoders to compress these diverse inputs into a coherent 2D latent representation.

3 Classification Analysis: Computer Vision on Fish Species

The first phase of our study focuses on the classification of marine species using visual data. This task represents a classic Computer Vision (CV) challenge where the goal is to map high-dimensional RGB input to a discrete set of semantic categories.

3.1 Dataset Analysis and Custom Implementation

The *Large Scale Fish Dataset* was initially analyzed to assess its suitability for deep learning. A critical step in ensuring model robustness is verifying class balance. By calculating the **Coefficient of Variation (CV)** across the class frequencies, we observed a value near zero ($CV \approx 0.02$), confirming that the dataset is highly balanced. This equilibrium prevents the model from developing a majority-class bias, ensuring that the loss function treats each species with equal importance.

To handle data loading, we opted against the standard `ImageFolder` utility in favor of a **custom FishDataset class** implemented in PyTorch. This decision was strategic: a custom implementation allowed for *fine-grained control* over the file system traversal. Specifically, it enabled us to programmatically filter out and ignore all "GT" (Ground Truth) sub-directories containing segmentation masks. This approach effectively eliminated the risk of data leakage, ensuring that the neural network learns to classify fish based on their physical features rather than relying on auxiliary binary masks.

3.2 Preprocessing and Normalization Pipeline

Before being fed into the network, raw images undergo a rigorous preprocessing pipeline to ensure numerical stability and computational efficiency:

- **Resizing:** All images are standardized to a resolution of 128×128 pixels. This resolution provides an optimal trade-off between the preservation of morphological details and the memory constraints of batch processing.
- **Tensor Conversion:** The pixel values are mapped from the $[0, 255]$ integer range to a floating-point range of $[0, 1]$.
- **Normalization:** We applied a normalization transform with a mean and standard deviation of 0.5 for all channels. This centers the data around zero, which is crucial for **stabilizing gradients** during the backpropagation process and accelerating the convergence of the optimizer.

3.3 CNN Architecture Design

The architecture follows a modular design inspired by the fundamental principles of *VGGNet* and *LeNet*, structured into a Feature Extractor and a Classifier Head.

The **Feature Extractor** consists of three consecutive **Convolutional Blocks**. Each block follows the pattern: `Conv2d` $\rightarrow$ `ReLU` $\rightarrow$ `MaxPool2d`. The convolutional layers utilize 3×3 kernels to capture local spatial patterns. As the data progresses through the blocks, the spatial dimensions are reduced by the Max-Pooling layers while the number of filters increases. This hierarchy allows the network to transition from detecting simple edges to identifying complex semantic features.

The **Classifier Head** receives the flattened output of the last convolutional layer. It consists of Fully Connected (FC) layers, integrated with a **Dropout layer (0.5)**. The inclusion of Dropout is essential for regularization, forcing the network to learn redundant representations and preventing over-reliance on specific neurons, thereby reducing the risk of overfitting.

3.4 Training Configuration

The training process was orchestrated using the PyTorch framework, providing explicit control over the computational graph.

- **Optimizer:** We utilized *Adam* (Adaptive Moment Estimation), chosen for its ability to adapt the learning rate for each parameter individually, which is particularly effective for deep architectures.
- **Loss Function:** The *Cross-Entropy Loss* was employed as the objective function, as it is the standard for multi-class classification tasks involving a softmax output layer.
- **Gradient Flow:** We implemented an explicit training loop, where gradients are computed via `loss.backward()` and updated using the `optimizer.step()` method, ensuring full transparency in the optimization process.

3.5 Results and Discussion

The performance of the CNN was evaluated through both quantitative metrics and qualitative analysis of the error patterns.

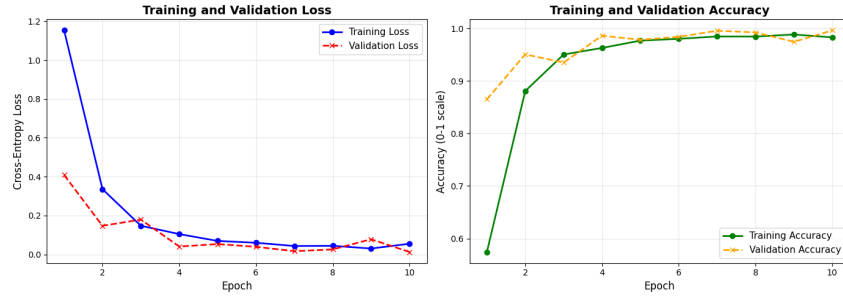


Figure 1: Training and Validation Accuracy/Loss curves.

As shown in Figure 1, the learning curves demonstrate a smooth convergence. The high *Validation Accuracy* ($> 95\%$) and its proximity to the training accuracy indicate that the model generalizes exceptionally well to unseen data without significant overfitting.

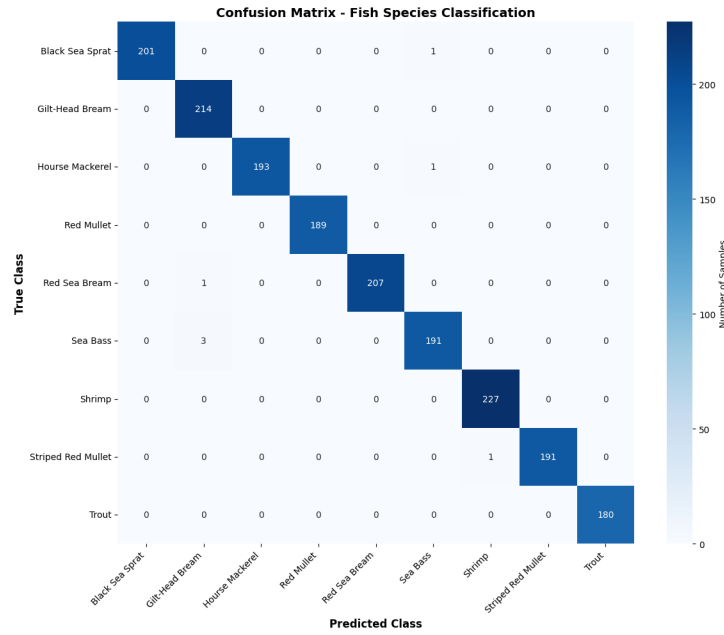


Figure 2: Confusion Matrix for the Fish Classification task.

A deeper qualitative analysis of the **Confusion Matrix** (Figure 2) reveals an important insight: false positives are not distributed randomly. Instead, they are strictly confined to

morphologically similar species, such as the *Sea Bass* being occasionally confused with the *Gilt-Head Bream*. This phenomenon is indicative of a "high-quality" error; it proves that the model has successfully learned relevant semantic features—such as body shapes, fin positions, and textures—rather than memorizing background noise or image artifacts. This reinforces the robustness of the Hybrid Framework Strategy in extracting meaningful patterns from visual domains.

4 Regression Analysis: Prediction of Used Car Prices

This section explores the prediction of vehicle prices using the *Audi Used Car Dataset*. The objective is to evaluate how different feature representation strategies (specifically linear versus non-linear dimensionality reduction) impact the performance of a Feed-forward Neural Network (FNN).

4.1 Dataset and Feature Engineering Strategy

The dataset comprises several thousand observations where the target variable is the *Price* of the vehicle. A primary challenge in this dataset is the high cardinality of the categorical feature `model`.

To mitigate this problem, we implemented a selective filtering strategy: we retained only the top 10 most frequent models and removed the remaining observations. This prevented the One-Hot Encoding process from generating an excessively sparse matrix, which would have hindered the model's ability to generalize. Further pre-processing steps were executed to reduce noise and optimize the dataset for the training phase.

The preprocessing pipeline was orchestrated using `ColumnTransformer`, allowing for parallel execution of:

- **StandardScaler**: Applied to numerical features (*year*, *mileage*, *engineSize*, *mpg*) to ensure zero mean and unit variance.
- **OneHotEncoder**: Applied to categorical features (*model*, *transmission*, *fuelType*) to transform them into a machine-readable binary format.

4.2 Dimensionality Reduction Architectures

We compared two distinct strategies to compress the input space from approximately 40 features to a fixed latent dimension of 8:

1. **Principal Component Analysis (PCA)**: Used as a linear baseline. It reduces dimensionality by projecting data onto the directions of maximum variance, assuming linear relationships between features.
2. **Autoencoder**: Implemented in Keras, this technique offers a distinct advantage over PCA by being non-linear, allowing it to capture complex feature relationships that linear methods might miss. The architecture is a symmetric Multi-Layer Perceptron (MLP):
 - **Encoder**: `Input → Dense(16, ReLU) → Bottleneck(8, Linear)`.
 - **Decoder**: `Bottleneck(8, Linear) → Dense(16, ReLU) → Output`.

The model was trained using **Mean Squared Error (MSE)** as the Reconstruction Loss, forcing the bottleneck to capture the most salient non-linear manifold of the data.

4.3 Hyperparameter Tuning via Cross-Validation

To ensure an objective comparison, we avoided heuristic parameter selection. Instead, we implemented a rigorous **K-Fold Cross-Validation** ($K = 5$) using `KFold`. This process was repeated independently for the three versions of the dataset: Original, PCA-transformed, and Autoencoder-compressed.

The grid search explored the following search space:

- **Learning Rate:** $\{10^{-2}, 10^{-3}\}$
- **Batch Size:** $\{32, 64\}$
- **Hidden Units:** $\{32, 64\}$ (variation in the depth and width of the FNN layers).

4.4 FNN Implementation and Metrics

The final regression model consists of an FNN with fully connected layers, **ReLU** activation functions, and the **Adam** optimizer. To evaluate the predictive performance and computational efficiency, we monitored four key metrics:

- **MAE (Mean Absolute Error):** For direct interpretability in terms of currency.
- **RMSE (Root Mean Squared Error):** To penalize significant outliers more heavily.
- **R^2 Score:** To measure the proportion of variance captured by the model.
- **Training Time:** Measured via the `time` library to assess the overhead of each approach.

4.5 Results and Comparative Discussion

The experimental results, summarized in Table 1, reveal a clear hierarchy in model performance.

Table 1: Performance comparison across different input representations.

Dataset	MAE (\$)	RMSE (\$)	R^2 Score	Train Time (s)
Original Data	1,652	2,450	0.94	43.3
Autoencoder	2,058	2,948	0.91	34.3
PCA	1,932	2,761	0.92	33.0

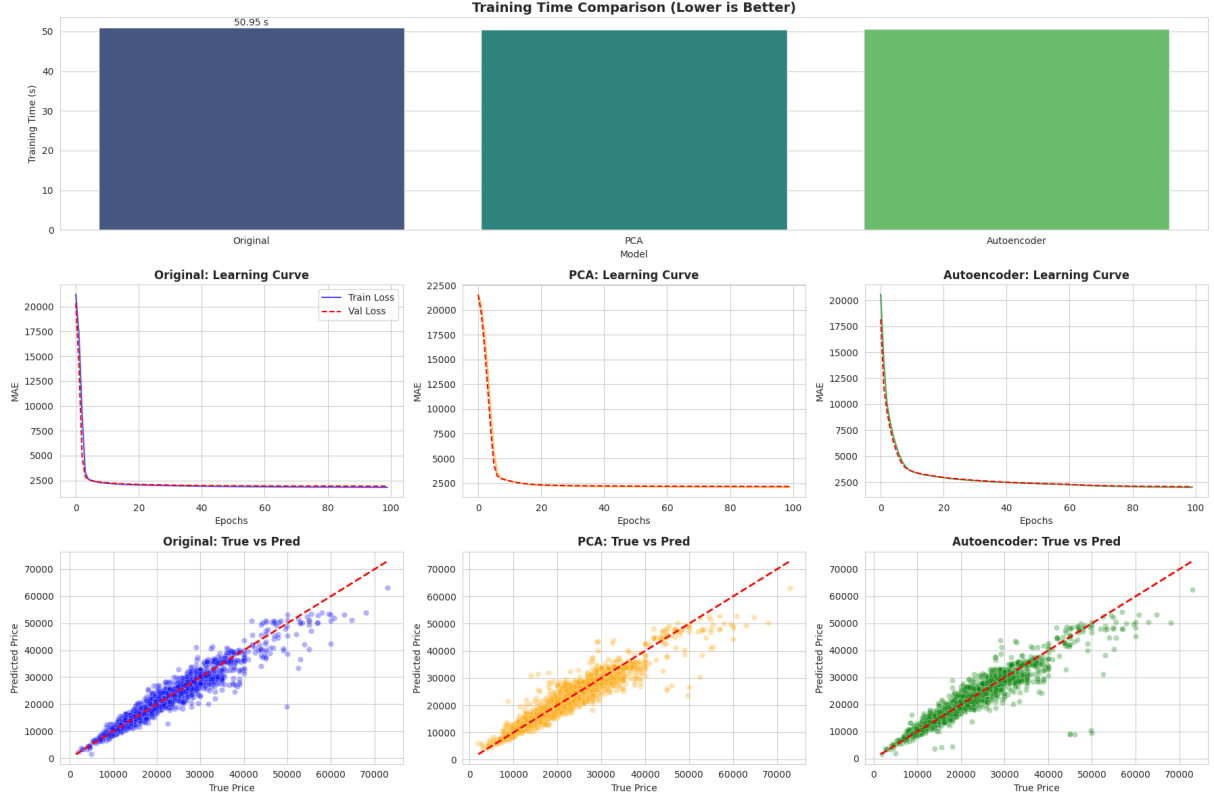


Figure 3: Comprehensive Regression Analysis: Comparison of performance metrics (R^2 , MAE), training time efficiency, and learning curves across Original, PCA, and Autoencoder implementations.

The **Original Dataset** yielded the best performance ($R^2 \approx 0.94$), as no information was discarded. However, Autoencoder and PCA also highly performed, with a R^2 score of 0.91 and 0.92). In terms of computational efficiency, as anticipated, the dimensionally reduced datasets performed better (33.0s and 34.3s), reducing the training time by approximately 10 seconds compared to the original dataset. This difference is due to the higher dimensionality and variability of the original feature space, which naturally requires more computational resources. However, visual analysis of the learning curves and scatter plots confirms that, overall, the model implemented on all three datasets achieved optimal convergence and performance results.

5 Clustering Analysis: Customer Segmentation

The third and final phase of our study transitions into unsupervised learning, with the goal of clustering via K-means: the objective here is to identify distinct customer profiles within the automotive sector. This task requires the model to discover latent structures in tabular data without the guidance of labels, presenting unique challenges in terms of evaluation and dimensionality.

5.1 Data Preprocessing and Feature Engineering

The *Customer Segmentation Dataset* was subjected to a rigorous cleaning and encoding process to ensure the compatibility of features with distance-based algorithms:

- **Cleaning:** Non-informative features such as **CustomerID** and **Category** were removed to prevent the introduction of noise. Duplicate entries and null values were addressed to maintain statistical integrity.

- **Encoding Strategy:**

- **Binary Encoding:** Features like *Gender*, *Married*, and *Graduated* were encoded through `OneHotEncoder`.
- **Ordinal Encoding:** *SpendingScore* was processed using an `OrdinalEncoder`. This was critical to preserve the **intrinsic hierarchy** (Low < Average < High), ensuring that the algorithm recognizes "Average" as a midpoint between the extremes.

- **Scaling:** A `StandardScaler` was applied via `ColumnTransformer`. Since K-Means relies on Euclidean distance, standardizing all features to zero mean and unit variance is mandatory to prevent features with larger scales from dominating the distance calculation.

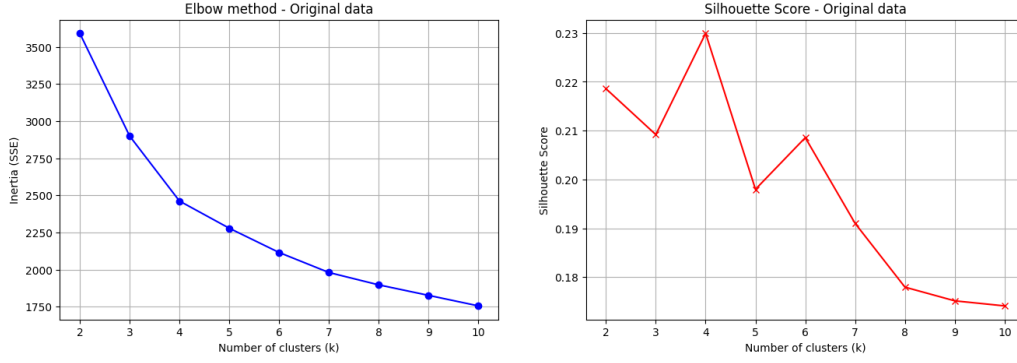
5.2 Dimensionality Reduction Strategy

To optimize the clustering process, we reduced the feature space to a **2D Latent Space** ($d = 2$) using both PCA and a Deep Autoencoder. This drastic reduction serves a dual purpose:

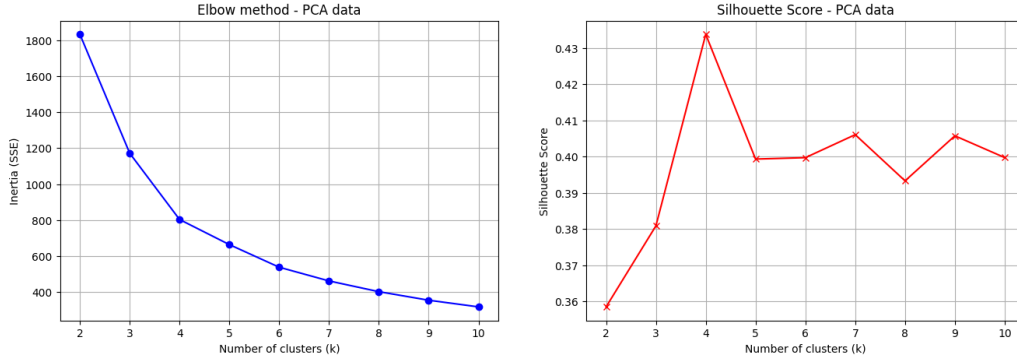
1. **Visualization:** Facilitating a direct interpretation of cluster separation on a Cartesian plane.
2. **Mitigation of the "Curse of Dimensionality":** In high-dimensional spaces, the ratio between the distance to the nearest and farthest neighbor tends toward 1, making the Euclidean distance effectively "meaningless." By projecting data into a 2D manifold, we enhance the discriminative power of the K-Means algorithm.

5.3 Optimal K Search: Elbow Method and Silhouette Analysis

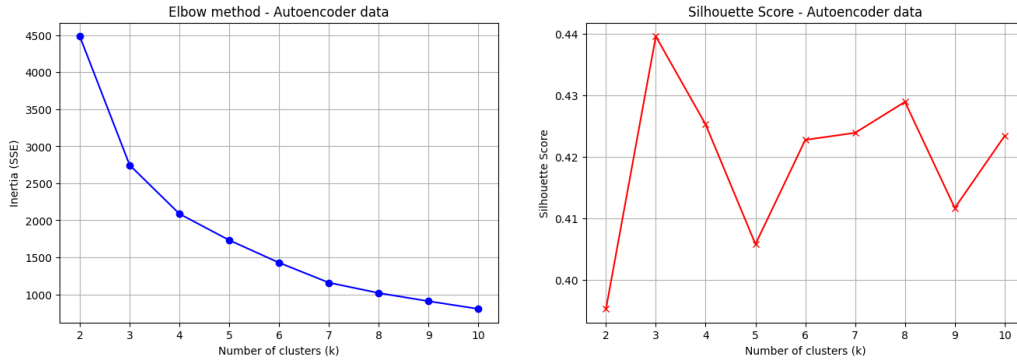
Determining the number of clusters (K) involved a comparative analysis of the **Elbow Method** (Inertia) and the **Silhouette Analysis** across the three data representations. This stage is crucial to evaluate how dimensionality reduction affects the structural clarity of the clusters.



(a) Metric Analysis on Original Data: Note the absence of a clear elbow and low silhouette scores.



(b) Metric Analysis on PCA-reduced Data (2D): The silhouette peak starts to emerge at $K = 4$.



(c) Metric Analysis on Autoencoder Latent Space (2D): Clearer elbow and highest silhouette stability.

Figure 4: Comparative evaluation of the Elbow Method and Silhouette Score. The transition from (a) to (c) highlights how non-linear dimensionality reduction clarifies the underlying cluster structure.

An in-depth analysis of the plots in Figure 4 reveals several technical insights:

- **Original Data (Fig. 4a):** The Inertia curve is almost linear after $K = 3$, exhibiting what we define as a "Soft Elbow". The Silhouette Score is extremely low (≈ 0.23), suggesting that in the high-dimensional space, the points are heavily overlapped or the Euclidean distance is suffering from the density of features.
- **PCA (Fig. 4b):** Reducing the data to two linear principal components significantly improves the separation. We observe a global maximum for the Silhouette Score at $K = 4$ (≈ 0.43). This indicates that the first two components contain enough variance to define four distinct groups.

- **Autoencoder (Fig. 4c):** The latent space generated by the neural network provides the most robust results. The Silhouette Score reaches its absolute maximum (≈ 0.44 at $K = 3$) and remains high at $K = 4$. The "elbow" in the Inertia plot is more pronounced compared to the original data, confirming that the **Autoencoder has successfully filtered out noise** and non-linear redundancies.

The ambiguity of the Elbow method, especially in the raw data, pushed us to **privilege the Silhouette Analysis**. Consequently, $K = 4$ was selected as the optimal number of clusters to ensure a balanced segmentation that remains consistent across all experimental setups.

5.4 Results

The results confirm that **latent space clustering is superior** to clustering in the raw feature space. The PCA-generated 2D space yielded the lowest Davies-Bouldin index (0.78), indicating more compact and better-separated clusters. Autoencoder also yielded optimal results in this sense.

The qualitative comparison (Figure 5) visualizes this improvement: while the original data produces overlapping "clouds" of points, Autoencoder in particular successfully disentangles the non-linear relationships between demographic and spending features, creating distinct segments. This proves that the non-linear manifold learning capability of the Autoencoder is highly effective at filtering out noise while retaining the fundamental structural information necessary for high-quality customer segmentation.

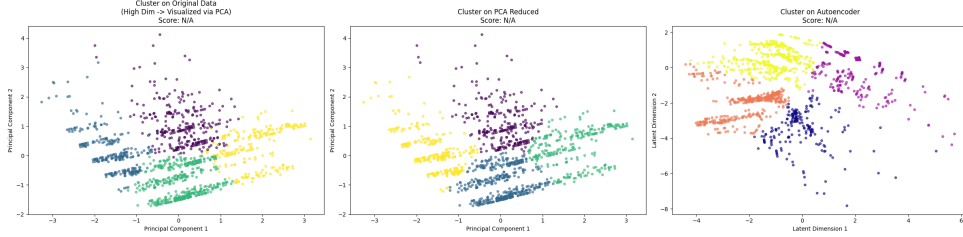


Figure 5: Visual comparison of K-Means results on different representations.

6 Final Conclusion

This project explored the application of supervised and unsupervised machine learning models across different data modalities, including images and tabular data.

6.1 Image Classification

In the image classification task, the Convolutional Neural Network trained on the fish dataset achieved very high accuracy on both training and validation data. The results confirm that CNNs are well suited for visual tasks, as they are able to automatically extract hierarchical spatial features from raw pixel data. The analysis of the confusion matrix showed that most misclassifications occurred between visually similar species, indicating that the model learned meaningful visual patterns rather than relying on background noise or spurious correlations.

6.2 Regression on Tabular Data

For the regression task on the Audi Used Car dataset, the best performance was obtained using the original feature space. This result is expected, as dimensionality reduction inevitably removes part of the available information. However, PCA and Autoencoder also obtained optimal results, proving that a regression task can also be implemented with a dimensionality reduction without

losing much. These results highlight the trade-off between model simplicity, training efficiency, and predictive accuracy when working with reduced feature spaces.

6.3 Clustering and Dimensionality Reduction

In the clustering task, dimensionality reduction played a crucial role in improving cluster quality. Applying K-Means directly to the original high-dimensional data resulted in poorly separated clusters and low silhouette scores. Both PCA and Autoencoder-based latent spaces improved clustering performance, but the Autoencoder produced the most stable and interpretable results. The reduced latent representation helped filter noise and irrelevant variations, allowing K-Means to better capture the underlying structure of the data.

This confirms that dimensionality reduction can be beneficial not only for computational efficiency, but also for enhancing the quality of unsupervised learning.

6.4 Final Remarks

Overall, the results of this project show that the choice of model and data representation strongly depends on the nature of the task and the structure of the data. Deep learning models proved essential for image-based problems, while for tabular data the benefits of non-linear representation learning became evident, especially in reduced feature spaces.

The use of different frameworks (PyTorch for computer vision and TensorFlow/Keras for tabular tasks) allowed us to select the most appropriate tools for each problem. This flexible approach made it possible to build clear and reproducible experiments while keeping the methodology aligned with the goals of the assignment.